



Basic E-Commerce Backend API

1. Project Overview

You have to build a **basic E-Commerce Backend API** using:

- **Node.js**
- **Express.js**
- **MongoDB**
- **Mongoose**

The application will have two types of users:

1. **Admin**
2. **User**

The main purpose of this project is to practice:

- REST APIs
- CRUD operations
- MongoDB
- Mongoose
- Authentication
- Authorization
- Middleware
- Filtering
- Pagination
- Role-based access control

2. User Roles

There should be **two roles** in the application.



Admin

Admin has **full access** to the application.

Admin can:

- Create products
- View products

- Update products
- Delete products
- Publish products
- Unpublish products
- View users
- Manage users

An important point:

An admin should be able to create a product and keep it **unpublished** until it is ready to be shown to users.

For example:

Admin creates:

iPhone 17

Price: ₹80,000

Category: Mobile

Published: false

Normal users should **not see this product**.

Later, admin publishes it:

Published: true

Now normal users can retrieve the product.

User

Normal users have limited access.

Users can:

- Register
- Login
- View published products
- View a single published product
- Filter products
- Sort products
- Filter products by category

- Filter products by price

Users **cannot**:

- Create products
 - Update products
 - Delete products
 - Publish products
 - Unpublish products
 - Manage other users
-

3. Database Requirement

There should be **only ONE product collection**.

Do **not** create separate collections such as:

- mobiles
- laptops
- shoes
- clothes

Instead, create one collection:

products

The category should be stored as a field inside the product.

For example:

```
{  
  "name": "iPhone 17",  
  "description": "Latest Apple smartphone",  
  "price": 80000,  
  "category": "Mobile",  
  "published": true  
}
```

Another product:

```
{  
  "name": "Nike Air Max",  
  "description": "Running shoes",  
  "price": 9000,  
}
```

```
"category": "Shoes",  
"published": true  
}
```

Both products exist in the **same collection**.

4. Product Fields

At minimum, a product should contain:

Product

```
|  
├── name  
├── description  
├── price  
├── category  
├── stock  
├── published  
├── createdAt  
└── updatedAt
```

Example:

```
{  
  "name": "Samsung Galaxy S26",  
  "description": "Samsung flagship smartphone",  
  "price": 75000,  
  "category": "Mobile",  
  "stock": 25,  
  "published": true  
}
```

You can add additional fields if required.

API Design Reference :

5. Product CRUD Operations

Admin should be able to perform all CRUD operations.

Create Product

POST /api/products

Only admin can access this API.

Get Products

GET /api/products

This API behaves differently depending on the user.

For a normal user:

Return only products where published = true.

For an admin:

Return all products, including unpublished products.

Get Single Product

GET /api/products/:id

Normal users should only be able to retrieve a product if it is published.

Admin can retrieve any product.

Update Product

PUT /api/products/:id

Only admin can update products.

Delete Product

DELETE /api/products/:id

Only admin can delete products.

6. Publish / Unpublish Product

Admin should have the ability to control whether a product is visible to users.

For example:

PATCH /api/products/:id/publish

or

PATCH /api/products/:id/unpublish

The product can have:

```
{  
  "published": true  
}
```

or

```
{  
  "published": false  
}
```

Important rule

If:

published = false

normal users **must not receive that product.**

7. Product Filtering

Users should be able to filter products.

For example:

Filter by category

GET /api/products?category=mobile

This should return products belonging to the mobile category.

Filter by price

GET /api/products?minPrice=10000&maxPrice=50000

This should return products between ₹10,000 and ₹50,000.

Combine filters

GET /api/products?category=mobile&minPrice=20000&maxPrice=60000

This means:

Give me published mobile products whose price is between ₹20,000 and ₹60,000.

8. Sorting

Users should be able to sort products.

For example:

Lowest price first

GET /api/products?sort=price_asc

Highest price first

GET /api/products?sort=price_desc

Newest products

GET /api/products?sort=newest

9. Pagination

Do not return 10,000 products in a single API response.

Implement pagination.

Example:

GET /api/products?page=1&limit=10

Response could look like:

```
{
  "products": [],
  "page": 1,
  "limit": 10,
  "totalProducts": 100,
  "totalPages": 10
}
```

10. Authentication

The application should have authentication.

Users should be able to:

Register

↓

Login



Receive authentication token



Access protected APIs

Passwords must **not** be stored directly in MongoDB.

Students should hash passwords using a library such as:

bcrypt

11. Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

For example:

Admin

└─ Create product	✓
└─ Update product	✓
└─ Delete product	✓
└─ Publish product	✓
└─ View unpublished	✓

User

└─ Create product	✗
└─ Update product	✗
└─ Delete product	✗
└─ Publish product	✗
└─ View published	✓

Students should implement this using **middleware**.

